VLB + ELB revise roadmap: rebuild the entry flow a new way

Date: 3 November 2026. Branch: `revise/vlb-elb` (cut from `main` at `07b8f46`, PR 64). Companion: `docs/vlb-stability-audit-2026-11-03.md` explains why the current build keeps breaking. This document is the plan to replace it.

Scope, in the order it will be built:

1. VLB manual entry
2. VLB scan through the software (scanner connector / local scanner)
3. VLB upload (PDF and photos)
4. ELB module (Equipment Log Book stage of the same batch)
5. Final synchronization (batch closure, Pulse trip verification, Pulse writes, reports)

---

1. Why a new way, in one page

The current VLB is not failing because of one bug. It fails because of how it is shaped:

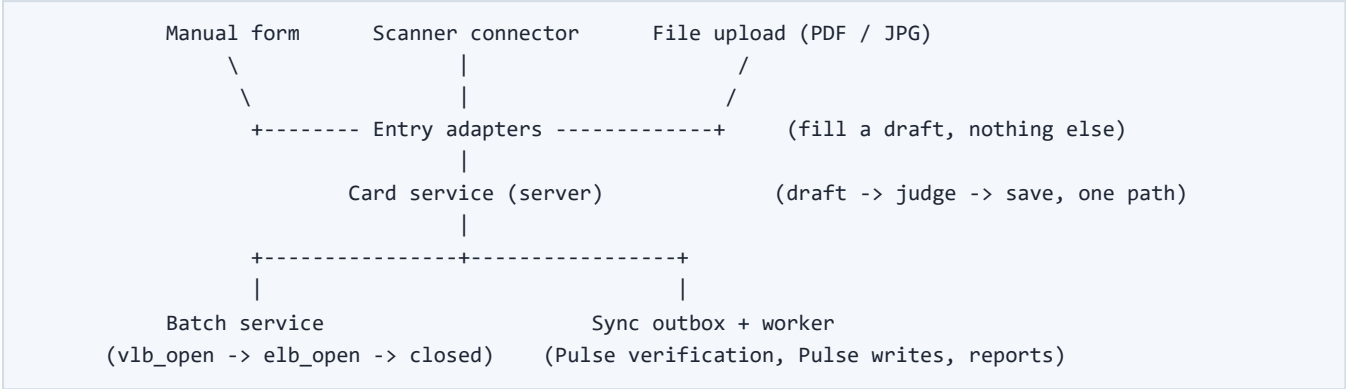
Problem today	Effect	New rule
Manual, scan and upload each have their own save path (`log-book.mjs`, `bulk-card-scanning.mjs`, auto-save in the browser)	A fix in one path is missing in the other two	**One card model, one save path.** Manual, scan and upload only differ in how the draft is filled.
The browser and the server each decide "is this card ready" with different master lists	Cards shown as ready are refused with 409; counters disagree (24 saved vs 35/37)	**The server is the only judge.** The browser shows the server's verdict; it never computes its own.
`src/bulk-card-scanning.jsx` is 93,000 characters on 333 lines; `src/main.jsx` is 950,000 characters	One edit silently drops a sibling control; four writers collide in one file	**Small files, one job each,** formatted at 120 columns, no file above about 400 lines.
282 tests read source text; a real crash passed all of them	Every refactor breaks tests; real defects are not caught	**Behaviour tests only** for the new code: call the function, call the route, render the component.
Every merge to `main` went live	Operators were the test team	**Test platform first, flag off in production,** one site pilot before everyone.

The new build is written **beside** the old one, in new folders, behind a flag. The old screens keep working for operators until the new path is proven site by site. Nothing is deleted until the last site has moved.

---

2. Target design

2.1 The four layers



- **Domain core** (`vlb/core/`, pure functions, no database, no React): card schema, normalisation, validation, the judge, machine resolution, closure checks. Everything else imports this.
- **Card service** (`vlb/server/`): the only code that writes a VLB or ELB card. Owns drafts, versions, the save transaction and the write into `mis_log_books`.
- **Entry adapters**: manual form, scan, upload. Each produces a draft and hands it to the card service. Extraction (Gemini, Groq, Paddle, local OCR) sits behind one interface.
- **UI** (`src/vlb/`): small components that render what the server returns. No business rule lives in the browser.

2.2 Folder layout

```
vlb/
  core/
    card-schema.mjs          VLB card, trip row, ELB card: fields, types, defaults
    normalise.mjs            door text, dates, times, meters, machine labels
    judge.mjs                judgeCard(card, masters) -> { verdict, issues[] }
    issues.mjs               issue codes + operator-facing text (one place)
    machine-resolve.mjs      label -> Equipment master id (wraps equipment-identity + aliases)
    batch-rules.mjs          canLockVlb, canCloseBatch, fleet status rules
    fingerprint.mjs          duplicate detection
  server/
    routes.mjs               /api/v2/vlb/* only, thin: parse -> service -> respond
    card-service.mjs         createDraft, patchDraft, saveCard, rejectCard, deleteCard
    batch-service.mjs        createBatch, lockVlb, unlockVlb, closeBatch, reopen
    masters-snapshot.mjs     one versioned master bundle per site per request
    extraction-queue.mjs     job table + worker (reuses existing readers)
    sync-outbox.mjs          outbox table + worker
    schema.mjs               additive SQL only
  adapters/
    manual.mjs  scan.mjs  upload.mjs
src/vlb/
  vlb-app.jsx                route shell, flag check
  batch-picker.jsx           site, date, shift
  card-board.jsx             tiles and counts from GET /board
  card-form.jsx              one form for every source, VLB and ELB
  trip-rows.jsx              trip grid
  upload-panel.jsx           drop zone + per-file progress
  scan-panel.jsx             scanner connector
  review-queue.jsx           next card needing attention
  elb-stepper.jsx            one machine at a time
  sync-status.jsx            closure and Pulse sync state
  api.mjs                    fetch wrapper with version + idempotency headers
test/vlb-v2/                 behaviour tests only
```

2.3 One card lifecycle for every source

State	Meaning	Set by
`draft`	Created, fields being filled (typed, scanned or uploaded)	adapter
`extracting`	Image is in the extraction queue	extraction worker
`needs_review`	Judge found issues; operator must act	card service
`ready`	Judge found no blocking issue	card service
`saved`	Written to `mis_log_books`, linked to batch	card service (single transaction)
`held`	Asset belongs to another site (express transfer pending)	card service
`rejected` / `deleted`	Operator removed it, with reason	card service

Rules that remove today's error classes:

- **Every write carries a version number.** `PATCH /cards/:id` with a stale version gets `409 stale` and the fresh card, never a silent overwrite. This ends the "my correction vanished after refresh" class.
- **Every save carries an idempotency key.** A double click, a retry after a timeout, or a browser reload re-sends the same key and gets the same answer. No duplicate log books.

- **Refusals are `422` with an issue list,** each issue with a code, field, row and text. The UI highlights the exact field. A `409` means only "someone else changed this".
- **Auto-save is the same call as Save.** The worker calls `saveCard` when the judge says `ready`; there is no second rule.
- **Counters come from one query.** `GET /api/v2/vlb/batches/:id/board` returns every tile count and the card list from the same rows. The browser never counts.

2.4 Data: additive, reports unchanged

Saved cards still go to `mis_log_books` with today's `data` shape, so production reports, MWS, Pulse trip verification and exports keep working without change. New tables only hold work in progress:

```
CREATE TABLE IF NOT EXISTS vlb_cards(
  id UUID PRIMARY KEY,
  batch_id UUID NOT NULL REFERENCES mis_vlb_batches(id),
  kind TEXT NOT NULL CHECK (kind IN ('vlb','elb')),
  source TEXT NOT NULL CHECK (source IN ('manual','scan','upload','stepper')),
  state TEXT NOT NULL,
  version INTEGER NOT NULL DEFAULT 1,
  data JSONB NOT NULL,                -- the card, in the core schema
  issues JSONB NOT NULL DEFAULT '[]',  -- last judge result
  confidence JSONB,                   -- per-field extraction confidence
  image_ref TEXT, fingerprint TEXT,
  logbook_id BIGINT REFERENCES mis_log_books(id),
  created_by TEXT NOT NULL, updated_by TEXT, created_at TIMESTAMPTZ DEFAULT NOW(), updated_at TIMESTAMPTZ
DEFAULT NOW());

CREATE TABLE IF NOT EXISTS vlb_idempotency(key TEXT PRIMARY KEY, response JSONB NOT NULL, created_at
TIMESTAMPTZ DEFAULT NOW());

CREATE TABLE IF NOT EXISTS vlb_extraction_jobs(
  id UUID PRIMARY KEY, card_id UUID NOT NULL REFERENCES vlb_cards(id),
  state TEXT NOT NULL CHECK (state IN ('queued','running','done','failed')),
  attempts INTEGER NOT NULL DEFAULT 0, reader TEXT, error TEXT, next_attempt_at TIMESTAMPTZ DEFAULT NOW());

CREATE TABLE IF NOT EXISTS vlb_sync_outbox(
  id BIGSERIAL PRIMARY KEY, batch_id UUID NOT NULL, logbook_id BIGINT,
  target TEXT NOT NULL,               -- 'pulse_trip_verification' | 'pulse_transfer' | 'pulse_equipment'
| 'batch_report'
  payload JSONB NOT NULL, idempotency_key TEXT NOT NULL UNIQUE,
  state TEXT NOT NULL DEFAULT 'pending' CHECK (state IN ('pending','sent','failed','blocked')),
  attempts INTEGER NOT NULL DEFAULT 0, last_error TEXT, next_attempt_at TIMESTAMPTZ DEFAULT NOW());
```

Batches keep `mis_vlb_batches` with the existing `stage` column (`vlb_open`, `elb_open`, `closed`) and the existing fleet and machine-snapshot tables from the two-stage build. The one-open-batch-per-site trigger is unchanged.

2.5 Flags

Flag	Default production	Default test platform	Meaning
`VLB_V2`	`off`	`on`	Shows the new VLB entry under its own menu item
`VLB_V2_SITES`	empty	`*`	Comma list of sites that use the new path; others stay on the old one

Flag	Default production	Default test platform	Meaning
`VLB_V2_AUTOSAVE`	`off`	`on`	Worker saves `ready` scan/upload cards without a click
`VLB_V2_SYNC`	`off`	`on`	Outbox worker sends to Pulse
`PULSE_WRITE_MODE`	`off`	`off`	Unchanged; `live` only when Pulse exposes its write endpoints

All flags are listed on `/api/health` so anyone can see what is on.

---

3. Branch and working rules

1. **`revise/vlb-elb` is the integration branch.** Each phase below is one or more short branches (`revise/p1-core`, `revise/p3-manual`, ...) opened as pull requests **into `revise/vlb-elb`**, never straight into `main`.
2. **Merge `main` into `revise/vlb-elb` every morning.** The new code lives in new folders, so conflicts should be rare; if one appears in an old file, the old file wins and the new code adapts.
3. **At the end of each phase, `revise/vlb-elb` merges to `main` with every flag off.** Production behaviour does not change. This keeps the branch from living for weeks and drifting.
4. **One writer per folder at a time.** Codex, Claude sessions and Caliber-tech may work in parallel only on different folders (for example core vs UI).
5. **Every pull request is reviewed by a session that did not write it,** checking: every `$n` placeholder and `\d` survived, every import resolves, the phase gate tests pass, and the smoke test in section 6 was run.
6. **No code through shell heredocs or `bash -c` strings.** Use the editor tools.
7. **Old VLB screens get crash fixes only** until cut-over. No new features on the old path.

---

4. Phases

Durations assume one focused developer (or one agent session plus your review). Each phase ends with a gate; do not start the next phase until the gate passes.

Phase 0. Stop the bleeding and prepare (day 1)

Step	Owner	Done when
Release current `main` (PR 61 and PR 64 fixes) to production through the release tag, after the ten-minute smoke test	You	Production `/api/health` shows `07b8f46` or later
Fix the release pipeline: Azure federated credential for the `release-*` tag subject (see PR 63)	You	One tag release succeeds end to end
Provision the test platform (`docs/test-platform-setup.md` sections 1.2 and 2–9): `caliber-mis-test-783`, scrubbed database copy, repo variables	You	Test URL opens and signs in
Freeze new features on the old VLB screens; crash fixes only	Everyone	Announced to all writers

Step	Owner	Done when
Close or merge stale PRs and delete merged branches and worktrees	You + one session	No more than 5 open branches touching VLB
Collect a **golden set**: about 200 real saved VLB cards and 50 ELB cards from last week, plus 30 scanned images with their corrected values	One session	Fixture files under `test/vlb-v2/golden/`, personal names scrubbed

Gate: production on latest main; test platform reachable; golden set committed.

> The test platform is the single biggest dependency. Without it, every later phase falls back to "production is the test", which is the cause of the current errors.

Phase 1. Domain core (days 2–4)

Build `vlb/core/` as pure functions with no database or React.

1. `card-schema.mjs`: one definition of a VLB card, a trip row and an ELB card. Field names match today's `mis_log_books.data` so saved output is identical.
2. `normalise.mjs`: move door text, date, time, meter and machine-label cleaning out of `scan-door-text.mjs`, `scan-meter-text.mjs`, `scan-card-date.mjs` by **importing** them, not copying. Only wrap.
3. `machine-resolve.mjs`: label to Equipment master id through `equipment-identity.mjs`, `scan-machine-rules.mjs` and equipment aliases (`V19 = VL19 = VPC 019`).
4. `issues.mjs`: every refusal reason in one catalogue with a code, severity (`block` / `warn`), and operator text. Today's texts ("Card still needs attention", "machine-summary conflict", meter limits, MWS rules from PR 64) become codes.
5. `judge.mjs`: `judgeCard(card, mastersSnapshot) -> {verdict: 'ready'|'needs_review'|'held', issues}`. Inputs are explicit; it cannot run without masters.
6. `batch-rules.mjs`: `canLockVlb(batch, cards)`, `canCloseBatch(batch, fleet, elbCards)`, fleet shift-status rules (Working needs hours > 0; a loader with trips must be Working or Working with partial breakdown).

Tests: table-driven, one file per module. The **golden test** runs `judgeCard` over every golden card and expects `ready` for every card production saved, and the recorded issue for every card production refused for a real reason.

Gate: golden test agrees with production on at least 98% of cards; every disagreement is listed and either fixed or accepted by you in writing.

Phase 2. Card service and API v2 (days 5–8)

1. `schema.mjs` with the additive tables in section 2.4, run by the existing migration runner.
2. `masters-snapshot.mjs`: loads Equipment, Shift, Employee, Region and aliases for one site once, gives it a version hash, caches for 60 seconds. Every judge call on the server uses it. `GET /api/v2/vlb/masters?site=` serves the same bundle to the browser for drop-downs only.
3. `card-service.mjs`: `createDraft`, `patchDraft(version)`, `judge`, `saveCard(idempotencyKey)`, `rejectCard`, `deleteCard`, `holdCard`. `saveCard` runs in one transaction: re-judge, insert or update `mis_log_books`, link the batch entry, set state `saved`, write audit. If the judge says no, nothing is written.

4. `batch-service.mjs`: wraps the existing stage logic (`vlb-batch-stages.mjs`, `vlb-two-stage.mjs`) behind the new rules; `lockVlb`, `unlockVlb`, `closeBatch`, `reopen` keep today's OTP and PM approval behaviour.
5. `routes.mjs` under `/api/v2/vlb/`:

Method and path	Purpose
`GET /batches?site=&date=`	Batch picker
`POST /batches`	Create batch (site, date, shift)
`GET /batches/:id/board`	Every tile count and the card list, one query
`POST /batches/:id/cards`	Create a draft (any source)
`GET /cards/:id`	Card, issues, image link, version
`PATCH /cards/:id`	Change fields, needs `If-Match: <version>`
`POST /cards/:id/save`	Save, needs `Idempotency-Key`
`POST /cards/:id/reject`, `DELETE /cards/:id`	Remove with reason
`POST /batches/:id/lock-vlb`, `/unlock-vlb`, `/close`, `/reopen-request`	Stage changes
`GET /batches/:id/sync`	Final sync status

The existing `assertVlbBatchWrite` and `assertElbBatchWrite` guards are called inside the service, so the old and new paths obey the same batch rules.

Tests: route tests against PGlite (already a dev dependency): create batch, create draft, patch with stale version (expect 409), save twice with same key (expect one log book), save an invalid card (expect 422 with issue codes, nothing written).

Gate: all route tests green; one saved card through v2 is byte-identical in `mis_log_books.data` to the same card saved through the old manual form.

Phase 3. VLB manual entry (days 9–12)

The simplest path, built first, because it proves the core and the card form that scan and upload will reuse.

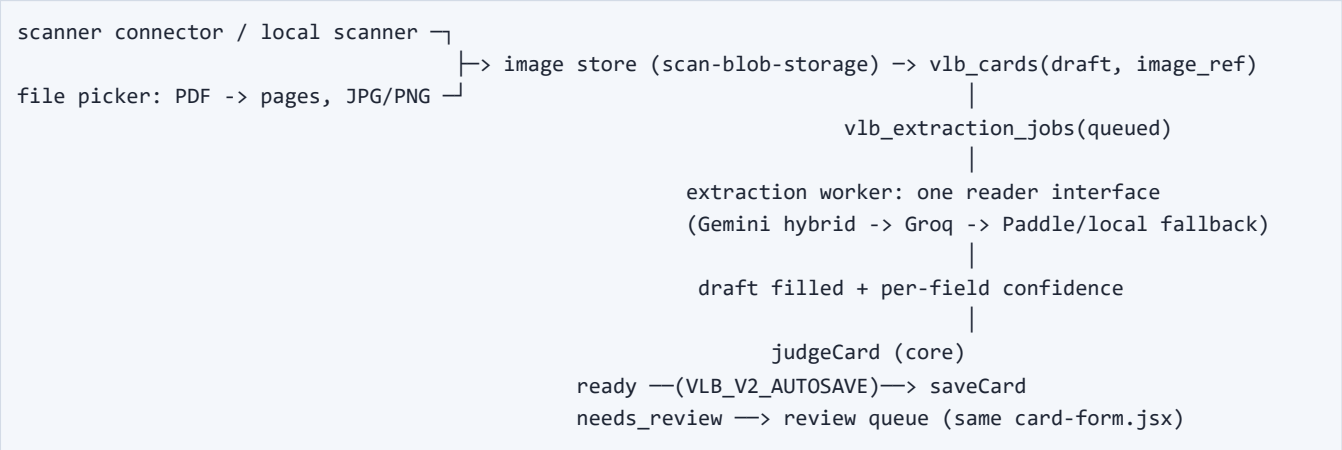
1. `batch-picker.jsx`: site (from the user's scope), date, shift; shows the open batch or offers to create one.
2. `card-board.jsx`: tiles (Draft, Needs review, Ready, Saved, Held, Rejected) and the card list, all from `/board`, refreshed after every action and every 10 seconds while visible.
3. `card-form.jsx`: header fields (door, driver, date, shift, opening and closing KMR/HMR) and `trip-rows.jsx`. Every field shows the server's issue text inline. Save is disabled only by `block` issues the server returned. Keyboard-first: Enter moves to the next field, the same as the current log book form.
4. Draft auto-save: each field change sends `PATCH` after 800 ms of no typing; a lost connection keeps the draft in the browser and resends on reconnect. A refresh or F5 reopens the same card.
5. Menu item **VLB entry (new)**, visible only when `VLB_V2=on` and the site is in `VLB_V2_SITES`.

Tests: component render tests (query by label: Save, Reject, Back to board, each field); one Playwright run on the test platform: create batch, enter three cards, refresh mid-card, save, see tiles move.

Gate: on the test platform, one operator (Anoop or Nitish) enters a full real shift of manual cards with no error report in `/api/client-errors`.

Phase 4. VLB scan through the software and upload (days 13–19)

Scan and upload share the same pipeline; only the source of the image differs.



1. `adapters/upload.mjs`: accept PDF and images; split PDF pages server-side (pdfjs is already a dependency); compute the fingerprint per page; a duplicate page shows on the board as **Duplicate** with Restore, the same rule as `SCAN_DUPLICATE_FINGERPRINT` today.
2. `adapters/scan.mjs`: wrap the existing scanner connector and local scanner panels; each scanned page becomes a draft exactly like an uploaded page.
3. `extraction-queue.mjs`: a table-backed queue with a fixed number of workers per site, retry with back-off (1, 5, 15 minutes), and a visible **Extraction failed, type it in** state. The browser never runs extraction and never holds the queue, so closing the tab loses nothing.
4. `reader.mjs`: one function `extractCard(image, kind) -> {fields, confidence, reader}` that calls today's readers (`gemini-hybrid.mjs`, `groq-card-ocr.mjs`, `paddle-card-ocr.mjs`, `local-card-ocr.mjs`). No new OCR work in this phase.
5. Review: `review-queue.jsx` opens the next `needs_review` card in `card-form.jsx` with the card image beside it, low-confidence fields highlighted, and **Save and next**. The same form as manual entry, so every manual fix is a review fix.
6. Equipment alias mismatch and the express transfer hold reuse `scan-alias-mismatch-dialog.jsx` and the existing OTP flow, called from the card form.

Tests: upload a 10-page golden PDF in a route test with a stub reader that returns the golden values: expect 10 drafts, the judged states from the golden file, and saved log books for the ready ones. Fault tests: reader throws, reader times out, the same PDF uploaded twice, server restart mid-queue (jobs resume).

Gate: on the test platform, a real shift's PDF (about 37 cards) goes from upload to all cards saved or in review with zero server refusals of a card the board showed as Ready, and the board counts match the saved log books exactly.

Phase 5. ELB module (days 20–25)

The ELB stage keeps the decisions already taken in `docs/vlb-elb-batch.md` (whole site fleet from Pulse, one dialog per machine, shift status rules, no ELB outside a batch). What changes is how it is built.

1. **Lock VLB** calls `canLockVlb` from the core: every VLB card saved, none held, every trip row's machine resolved. It builds the machine snapshot and the fleet list (existing tables) inside one transaction.

2. `elb-stepper.jsx`: one machine per screen with Next, Previous, Skip. Each machine is a `vlb_cards` row of `kind='elb'`, `source='stepper'`, so it gets drafts, versions, idempotent saves and the same judge. Trips loaded are read-only from the snapshot. Previous shift's closing HMR proposed as opening HMR.
3. ELB scan and upload: the same pipeline as phase 4 with `kind='elb'`. A scanned ELB fills the matching machine's row.
4. **Close batch** calls `canCloseBatch`: every fleet machine has a saved ELB with a shift status; no held card; no duplicate ELB per asset. The refusal lists machines by name, loaders first with their trips.
5. Unlock VLB and reopen keep today's PM approval and OTP behaviour.

Tests: core tests for every shift-status rule; route tests for lock, unlock, close and reopen; a full-batch scenario test from golden data (VLB cards in, lock, ELB for every machine, close).

Gate: on the test platform, one real shift goes from VLB to ELB to closed; the machine-wise report (trips, HMR, trips per hour) matches a hand count from the paper cards.

Phase 6. Final synchronization (days 26–30)

Today sync is spread over several timers and is invisible to the operator. The new build makes it one outbox with a visible status.

1. **On close,** the batch service writes outbox rows in the same transaction as `closed_at`: one `pulse_trip_verification` per saved VLB, one `batch_report` per batch, and `pulse_transfer` / `pulse_equipment` rows for any express transfer or new asset approved in the batch.
2. `sync-outbox.mjs` worker: picks pending rows, sends with the row's idempotency key, retries with back-off, marks `failed` after 8 attempts and alerts on Telegram (existing `vlb-batch-alerts.mjs`).
    - `pulse_trip_verification` calls the existing `pulse-trip-verification.mjs` logic.
    - `pulse_transfer` and `pulse_equipment` call `pulse-writer.mjs`. While `PULSE_WRITE_MODE=off` these rows are `blocked` and appear in the existing Pulse pending list, so nothing is silently "local-only".
    - `batch_report` generates and stores the batch PDF and the WhatsApp/Telegram summary.
3. `sync-status.jsx` on the batch: **Synced 37 of 37 · 2 waiting on Pulse · 0 failed**, with the reason per row and a Retry button for administrators.
4. **Reopen** marks the batch's sent rows as superseded and writes new rows on the next close, so Pulse never holds a stale version.
5. **Nightly reconciliation**: for each batch closed yesterday, compare MIS log books with Pulse trip verification results and list differences on the diagnostics page.

External dependency: Pulse must expose the transfer-complete and equipment-create endpoints that `pulse-writer.mjs` expects. Until then, express transfers and new assets stay `blocked` and are done by the Pulse administrator, as today.

Tests: outbox worker with a fake Pulse that fails twice then succeeds (expect one delivery, not three); a crash between write and acknowledgement (expect the idempotency key to prevent a duplicate); reopen then close (expect superseded rows).

Gate: three consecutive closed shifts on the test platform show **Synced** for every row except the documented Pulse-blocked ones.

Phase 7. Pilot and cut-over (days 31–45)

1. Merge `revise/vlb-elb` to `main` with all flags off. Release to production.

2. **Shadow week** (production, one site): `VLB_V2_SITES=<pilot site>` with `VLB_V2_AUTOSAVE=off`. Operators use the new screen; a daily job compares the new judge's verdict with what the old judge would have said and lists every difference.
3. **Pilot week:** autosave on for the pilot site. Daily check of `/api/client-errors`, sync status and the comparison report.
4. **Roll-out:** add one or two sites per week to `VLB_V2_SITES`. Rollback for a site is removing it from the list; no release needed.
5. **Retire the old path** once every site has run two clean weeks: remove the old VLB menu items, then `src/bulk-card-scanning.jsx`, the VLB parts of `src/main.jsx`, the old scan routes and the source-text tests that only covered them. One pull request per removal.

Gate for each new site: previous sites had zero refusals of Ready cards and zero failed sync rows for a full week.

---

5. Timeline at a glance

Week	Phases	Visible result
1 (3–8 Nov)	0, 1	Production stable on latest fixes; test platform up; core judge agrees with production on the golden set
2 (9–15 Nov)	2, 3	New manual VLB entry working on the test platform
3 (16–22 Nov)	4	Scan and upload on the test platform, a real shift's PDF end to end
4 (23–29 Nov)	5	ELB stage and batch closure on the test platform
5 (30 Nov–6 Dec)	6	Final synchronization with visible status
6–8 (7–27 Dec)	7	Pilot site, then one or two sites a week, then retire the old path

---

6. Ten-minute smoke test (every pull request into `revise/vlb-elb`, every test-platform deploy)

1. Open `/api/health`; check the commit and the flag list.
2. Create a batch for a test site, date and shift.
3. Enter one manual VLB card with two trip rows; press F5 mid-card; confirm the draft is back; save.
4. Upload a three-page PDF; wait for extraction; confirm the board counts equal the cards.
5. Open one card in review; correct a field; Save and next.
6. Try to save a card with a missing door: expect the field highlighted, no 409.
7. Lock VLB; save one ELB machine in the stepper; skip one; try Close: expect the skipped machine named.
8. Finish the ELB; Close; open sync status.
9. Open `/api/client-errors`: expect nothing new.

---

7. Decisions needed from you

Decision	Recommendation	Blocks
Who provisions the test platform and the release credential, and by when	You, by 5 November; the runbook is ready	Every phase gate
Pilot site for phase 7	The site with the most scanned cards and a PM who answers OTPs quickly	Phase 7
Whether the new manual form replaces the old VLB manual form or sits beside it during the pilot	Beside it, behind the menu flag	Phase 3
Pulse write endpoints: who asks the Pulse team and for which date	Raise now; phase 6 ships with them `blocked` if not ready	Phase 6 Pulse writes only
Which writers (Codex, Claude sessions, Caliber-tech) own which folder	Core + server to one writer, UI to another	Phase 1 onward

8. What this roadmap deliberately does not do

- It does not rewrite OCR or extraction. The readers are reused behind one interface; accuracy work is a separate track.
- It does not change the two-stage business rules already agreed in `docs/vlb-elb-batch.md`.
- It does not change reports. Saved cards keep the `mis_log_books` shape reports read today.
- It does not touch other modules in `src/main.jsx`.